CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Project Charter

Team 20

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

> This is the final version of the charter submitted earlier in the semester. The fields below expand on that submission; nothing in it has been reversed. Two points of wording are worth stating plainly, since the submitted charter is the document on record:
>
> - The submitted charter describes an "**AI-assisted**" dashboard. AI assistance was applied to building the tool, and is logged in full in `docs/05-ai-usage-log.md`. The delivered tool calls no language model at runtime; the plain-language commentary is rule-based and deterministic, for the reasons set out under *Scope boundaries* below.
> - The submitted charter lists PV, EV, AC, CPI, SPI, EAC, ETC and TCPI. All are implemented. The tool additionally reports CV, SV, Earned Schedule — SV(t), SPI(t) and IEAC(t) — and all four standard EAC methods rather than one.

Item	Details
Team number	20
Team members and roles	**Sajjad Nakhwa** (23B0702) — Project Lead and Finance Modeller: owns the EVM method, the choice of forecasting formulas and the index conventions used throughout. **Dependar Arya** (23B0331) — Data Lead: owns the case study project, the work breakdown structure, the time-phased budget and the source list. **Neel Shah** (23B4244) — AI and Tool Developer: owns the Python engine, the Streamlit dashboard and the Excel export; manages the repository. **Akshat Karkar** (23B4229) — Validation Engineer: owns the manual benchmark workbook, the hand calculations, the test suite and the discrepancy log. **Pearsh Gumasta** (23B4232) — Documentation and Reflection Lead: owns the user manual, the AI usage log, the reflection report and the presentation.
Topic selected	Topic 7 — Earned Value Management (EVM) Dashboard
Problem statement	Construction project managers learn that a project is in trouble too late to act. Monthly reporting typically compares money spent against money budgeted, which conflates two different failures: spending more than planned, and doing less work than planned. A project that has spent 28% of its budget may be 28% complete or 20% complete, and the cost report alone cannot distinguish them. Earned Value Management separates the two by introducing a third measure, the budgeted cost of the work actually done, and from it derives early warning indices. The problem this tool addresses is that EVM is computationally tedious at work-package level, is therefore usually either skipped or rolled up to a single project curve that cannot locate a variance, and produces indices that non-specialist decision-makers cannot interpret.
Intended users of the tool	Primary: a project manager or planning engineer preparing a monthly progress report for a contractor or a client. Secondary: a client's project management consultant reviewing contractor claims; a lender's independent engineer assessing whether a project will need more funding than sanctioned. Each of these reads indices rather than computing them, which is why the tool leads with a plain-language verdict rather than a table.
Methods to be applied	Earned Value Management per the standard definitions: the base measures PV, EV and AC; variances CV and SV; the performance indices CPI and SPI; the four standard estimate-at-completion methods; ETC, VAC and TCPI. Extended with **Earned Schedule** (Lipke, 2003), which re-expresses schedule performance in time rather than currency and removes the well-known defect by which SPI returns to 1.00 at completion however late a project finishes. Earned value is measured by the **percent-complete method** at work-package level.

Item	Details
Data sources	A synthetic case study project designed for the purpose: a G+4 RCC academic block, BAC ₹14.00 crore, 12-month baseline, 19 work packages, reported monthly with a data date at the end of Period 6. Budgets are order-of-magnitude plausible for an institutional building of this size built to CPWD DSR-style item rates. **All figures are assumed and are not drawn from any tender, estimate or invoice.** The reasoning behind the choice of synthetic over real data is set out in the method note.
Build platform	Python 3.12 for the calculation engine; Streamlit with Plotly for the dashboard; openpyxl for the Excel export and the manual benchmark workbook; pytest for the test suite; LibreOffice in headless mode to evaluate the benchmark workbook's formulas during validation. All source is in a public Git repository.
Benchmark case for validation	All EVM indices for **reporting periods 5 and 6** of the case study, computed independently in a plain Excel workbook in which every figure is a typed formula acting on the raw inputs. The two implementations — Python and Excel formulas — are compared measure by measure. The Excel side is *evaluated* by a real spreadsheet engine rather than re-implemented in Python, because a Python re-implementation would only confirm our own understanding of the formulas against itself.
Key risks to completion	**1. Misunderstanding a formula consistently.** The most dangerous failure mode: an error made identically in both the tool and the hand check passes validation. Mitigated by writing the hand calculation from the textbook definitions before the engine, and by asserting algebraic identities (TCPI against EAC1 must equal CPI; VAC on EAC2 must equal CV) that hold only if the whole chain is correct. **2. Plausible-looking wrong data.** A case study tuned to produce interesting indices can quietly become internally inconsistent. Mitigated by strict input validation that raises rather than warns, and by reconciliation checks that the time-phased budget sums to each package budget and to BAC. **3. Over-reliance on AI-generated code.** Mitigated by the validation strategy above and by a test suite whose expected values are hand-derived, not generated. **4. Scope creep into schedule analysis.** EVM touches CPM scheduling, and modelling the network would not have been finishable. Explicitly out of scope; the tool consumes a time-phased budget and does not compute a critical path.

Scope boundaries

Stating what the tool does **not** do is as much a part of the charter as what it does, because every omission below is a place where a user could reasonably expect more.

In scope

- Work-package-level EVM, so that a variance can be attributed rather than merely detected.
- All standard indices, variances and the four standard EAC methods.
- Earned Schedule and time-based schedule indices.
- Plain-language monthly commentary generated deterministically.
- Export to a live-formula Excel workbook that recalculates when edited.

Out of scope

- **Critical path and network analysis.** The tool reads a time-phased budget. It does not know which activities are on the critical path, and therefore cannot say whether a given slip delays completion.
- **Re-baselining.** One baseline per project. A project that has been formally re-baselined needs the new baseline loaded as a new project.

- **Resource and cash flow modelling.** No retention, no mobilisation advance, no interest on overdraft. These matter enormously to a contractor and are the subject of a different topic in this brief.
- **Variation and claim management.** Scope changes are assumed to have been incorporated into the budget before the data reaches the tool.
- **Multi-currency and escalation.** All figures are nominal rupees at one price base.

The limitation that governs everything else

Earned value is percent complete multiplied by budget. Percent complete is a judgement made at site by a person with an interest in the answer.

If a work package is reported at 60% complete when it is really 40%, the tool will compute a confident, precise, internally consistent and entirely wrong earned value, and every index derived from it inherits the error. Validation against a manual benchmark proves the *arithmetic* is right. It cannot prove the *measurement* is right, and nothing in software can.

This is stated in the charter, repeated in the user manual, repeated on the dashboard's data tab, and repeated in the exported workbook, because it is the single thing a user most needs to carry away and the single thing a polished interface is most likely to obscure.